Turning a Bangla music VAE workshop paper into a conference submission

Skip EUSIPCO 2026 and SMC 2026 — both deadlines are already closed. Target ISMIR 2026 (full paper due April 27, 2026, AoE). That leaves roughly a week from today to extend the workshop paper. The two additions you proposed — real Bangla lyrics and a pretrained audio-encoder baseline — are exactly what ISMIR reviewers currently expect from multimodal / non-Western MIR submissions, and both are achievable overnight on CPU. The highest-leverage plan: (1) pull ~12–15k Bangla lyrics from Kaggle + BanglaMusicStylo + Wikisource Bengali (no scraping, no API quotas), match to BanglaBeats with RapidFuzz, and encode with `paraphrase-multilingual-MiniLM-L12-v2`; (2) extract frozen **LAION-CLAP** and **MERT-v1-95M** embeddings for all 20,020 clips (~3 h total on CPU), run K-Means as drop-in baselines, and retrain your VAE on CLAP/MERT features as an additional experiment. Drop wav2vec2 unless reviewers specifically demand a speech-SSL control — it was trained on LibriSpeech and transfers poorly to music, arXiv which 2024–2025 ISMIR papers consistently show.

Below is the end-to-end implementation playbook.

---

1. Conference deadlines — only ISMIR 2026 is still open

Venue	Location	Paper deadline	Page limit	Format	Status
ISMIR 2026	Abu Dhabi, Nov 8–12	**Abstract Apr 20 · Full paper Apr 27, 2026 AoE**	**6 + unlimited refs/ethics**	Double-blind, mandatory LaTeX template	**Open**
EUSIPCO 2026	Bruges, Aug 31–Sep 4	Feb 13, 2026	5 (incl. refs)	IEEE, single-blind	**Closed**
SMC 2026	Zagreb, Nov 5–7	Mar 13, 2026	8 (6 encouraged)	SMC/MBZ template	**Closed**

ISMIR's 2026 topic list explicitly names *"lyrics and other textual data; multimodality; musical style and genre; computational ethnomusicology"* ismir — a near-perfect fit for cross-lingual Bangla music clustering. If the April deadline slips, aim for the **ISMIR 2026 Late-Breaking/Demo track** (CFP typically posts in Aug/Sep, 2+1 pages, lightly reviewed) or a co-located workshop. EUSIPCO 2027 is the next signal-processing cycle.

Formatting implications for the extension: ISMIR's "6 + N" format is generous — put the related-work, dataset, and ablation detail into the scientific 6 pages, and let the references section expand freely. Plan for a new **Table 1** comparing MFCC-VAE, CLAP+K-Means, MERT+K-Means, and (optionally) VAE-on-CLAP; a **per-language (EN vs BN) breakdown**; and a **lyrics-present subset ablation**. Reviewers at ISMIR will also look for t-SNE/UMAP figures of the latent space

and a language-confound check showing your clusters aren't just separating English from Bangla.

---

2. Task 1 — Real Bangla lyrics: sources, pipeline, coverage

2.1 The best free sources, in priority order

Stop with Genius as your primary source. Empirically Genius is Hindi/English-biased; for the Bangla artists in your dataset (Arijit Singh Bengali-film tracks, Anupam Roy, Rabindra Sangeet, Nazrul Geeti, James/Bangladesh, Miles, Artcell, Warfaze, Shironamhin) coverage is sparse and, when present, often **romanized rather than Bangla script**. Expect ≤5–15% hit rate and further attrition after filtering for native script. Use Genius as a gap-filler, not a primary.

The highest-yield, lowest-friction stack:

1. **Kaggle bulk dumps (no API):** `shakirulhasan/bangla-song-lyrics` (~4k native-script lyrics) GitHub github + `meherabhasansajid/bangla-song-lyrics-dataset-with-genres-and-artists`. Download once, store as CSV.
2. **BanglaMusicStylo** (Hossain et al., ICBSLP 2018) — 2,824 lyrics across 211 lyricists IEEE Xplore +2 on GitHub + IEEE DataPort; organized by author, IEEE DataPort already clean Bangla Unicode.
3. **Rabbi et al. 2024 "Genre Classification from Bangla Music Lyrics"** — ~10,500 lyrics across 11 genres, Semantic Scholar released with the paper. Semantic Scholar This is the single largest Bangla-lyric corpus published to date and is directly genre-labeled.
4. **Wikisource Bengali** (`bn.wikisource.org`) — hosts the complete Gitabitan (~2,000 Tagore songs) and most of Nazrul's oeuvre in clean Unicode under **CC-BY-SA 4.0**; *republishable* because Tagore (d. 1941) and Nazrul (d. 1976) Shuddhashar are in the public domain in Bangladesh and India.
5. **LyricsGenius** for a targeted sweep over *unmatched* popular artists only.
6. **Targeted scraping** of `banglasonglyrics.com` (an open-source GPL-3.0 scraper already exists: `KhanShaheb34/BanglaSongLyricsScraper`), GitHub github `lyrics71.net`, `lyricsbengali.in` as a final fill, at ≥2s/request with identifying User-Agent.

Combined and deduplicated, priorities 1–4 alone yield roughly **12–15k unique Bangla lyrics in native script** before you ever touch a scraper. Hugging Face has **no Bangla song-lyrics dataset** as of April 2026; AI4Bharat's IndicCorp/Samanantar/MuRIL are general Bangla text, not song-specific.

Legal/ToS posture for the paper: follow the 4MuLA convention — release **embeddings, SHA-256 hashes, and source URLs**, never redistribute raw lyrics. Add a DMCA contact in the dataset README. This is standard MIR practice (used by Million Song Dataset, Laurier et al., Delbouys et al.) and is not a reviewer concern when handled this way.

2.2 Genius / LyricsGenius — as a gap-filler

```python
# pip install lyricsgenius
import os, json, lyricsgenius
genius = lyricsgenius.Genius(
    os.environ["GENIUS_TOKEN"],          # from genius.com/api-clients
    sleep_time=1.0, retries=3, timeout=15,
    remove_section_headers=True, skip_non_songs=True,
    excluded_terms=["(Remix)", "(Live)", "Instrumental"],
    verbose=False,
)

def fetch(title, artist):
    try:
        song = genius.search_song(title=title, artist=artist)
    except Exception as e:
        return {"title": title, "artist": artist, "error": str(e)}
    return None if song is None else {
        "title": song.title, "artist": song.artist,
        "url": song.url, "lyrics": song.lyrics, "id": song.id,
    }

# CRITICAL: write with ensure_ascii=False so Bangla isn't escaped as \u09xx
with open("genius_dump.json", "w", encoding="utf-8") as f:
    json.dump([fetch(t, a) for t, a in unmatched_pairs],
              f, ensure_ascii=False, indent=2)
```

Genius's API has **no officially documented rate limit** — don't cite one in the paper. 1 req/s with retries is community convention. GitHub Readthedocs The lyric text itself is HTML-scraped from the song page by LyricsGenius; Readthedocs store locally, don't redistribute.

2.3 Bangla script handling and MiniLM compatibility

`paraphrase-multilingual-MiniLM-L12-v2` inherits the **XLM-R SentencePiece tokenizer**, whose 250k vocab explicitly includes Bengali Unicode (and separately Bengali-Romanized) per Conneau et al. 2020 Table 6. arXiv So **Bangla script is natively tokenized, no byte fallback**. The caveat: MiniLM's paraphrase distillation training didn't guarantee Bengali supervision, and empirical quality on Bangla is visibly weaker than on Hindi/Arabic. If publication quality on the text side matters, consider upgrading to **`paraphrase-multilingual-mpnet-base-v2`** (same API, ~3× size, notably better Bengali) or **`LaBSE`** (109 languages, Google Research supervised parallel, generally the strongest low-resource option). Sentence Transformers

Convert everything to Bangla Unicode before encoding. Romanized ("Banglish") forms live in a much sparser slice of XLM-R's vocabulary and semantic similarity between the two is

unreliable. Use `bnbphoneticparser` for Avro-style loose romanization and `indic-transliteration` (ITRANS) for strict romanization.

```python
# pip install bnlp-toolkit bnunicodenormalizer bnbphoneticparser indic-transliterati
import re
from bnlp import CleanText
from bnunicodenormalizer import Normalizer
from bnbphoneticparser import BanglishToBengali
from sentence_transformers import SentenceTransformer

bn_norm, b2b = Normalizer(), BanglishToBengali()
clean = CleanText(fix_unicode=True, unicode_norm=True, unicode_norm_form="NFKC",
                  remove_url=True, remove_email=True, remove_emoji=True,
                  remove_punct=True)
BN_RANGE, LATIN, WS = re.compile(r'[\u0980-\u09FF]'), re.compile(r'[A-Za-z]+'), re.c

def bangla_ratio(t):  # fraction of non-whitespace chars in Bengali block
    return len(BN_RANGE.findall(t)) / max(len(re.findall(r'\S', t)), 1)

def clean_bangla_lyrics(text):
    if bangla_ratio(text) < 0.30 and re.search(r'[a-z]', text.lower()):
        text = b2b.parse(text)                        # romanized → script
    t = clean(text)
    t = " ".join((bn_norm(w) or {"normalized": w})["normalized"] for w in t.split())
    t = WS.sub(" ", LATIN.sub(" ", t)).strip()
    return t

st = SentenceTransformer("sentence-transformers/paraphrase-multilingual-MiniLM-L12-v
def encode_lyric(text):  # returns 384-d L2-normalized vector
    return st.encode(clean_bangla_lyrics(text), normalize_embeddings=True)
```

2.4 Matching BanglaBeats tracks to lyric entries

Use **RapidFuzz** `token_set_ratio` (order-invariant, robust to "feat.", "(Official)", version suffixes) with threshold **80–85**. Build candidate keys in both scripts to handle the fact that BanglaBeats metadata is inconsistent across artists.

```python
```

```
# pip install rapidfuzz
from rapidfuzz import process, fuzz
from indic_transliteration import sanscript
from indic_transliteration.sanscript import transliterate

def canon(s):
    s = re.sub(r"[\(\[].*?[\)\]]", "", s.lower())
    s = re.sub(r"[^\u0980-\u09FFa-z0-9 ]", " ", s)
    return re.sub(r"\s+", " ", s).strip()

def keys(title, artist):
    k = [f"{canon(title)}|{canon(artist)}"]
    if re.search(r"[A-Za-z]", title):
        try: k.append(f"{canon(transliterate(title, sanscript.ITRANS, sanscript.BENG
        except Exception: pass
    return k

def match(qkeys, corpus_keys, thr=82):
    best = None
    for q in qkeys:
        hit = process.extractOne(q, corpus_keys, scorer=fuzz.token_set_ratio, score_
        if hit and (best is None or hit[1] > best[1]): best = hit
    return best
```

Manual spot-check the 500 borderline matches (score 78–85). Expect 20–40% of your Bangla subset to match, i.e. **roughly 2,500–6,000 of 20,020 tracks** if ~50% are Bangla. Run a 500-track pilot before committing to the full pipeline.

2.5 How many real lyrics is "enough" for ISMIR?

There is no reviewer-published threshold. The honest answer is: there is no rule, only conventions. What surveyed ISMIR/EUSIPCO multimodal papers actually do (Laurier 2008, Neumayer & Rauber 2007, Delbouys 2018, 4MuLA 2020, Muchomusic 2024):

- For a **primary claim** that fusion helps on *all* 20k tracks → you'd need ≥50% lyrics coverage, or you must restrict the claim to the matched subset.
- For a **secondary ablation** ("lyrics improve clustering where available") → **10–25% is accepted**, provided the paper reports results separately on the lyrics-present subset and the full set and uses a proper missing-modality fallback.
- 4MuLA (96k songs, partial lyrics coverage) ACM Digital Library is the best precedent — partial is fine, transparency is mandatory.

Frame the extension as a **lyrics-aware ablation on the matched subset** rather than as a claim over the entire dataset. Report: N_total, N_lyrics_matched, per-genre match rates, and clustering

metrics both overall and on the lyrics-present subset.

2.6 Missing-modality fallback without label leakage

The failure mode: if all tracks lacking lyrics share a zero or `[no_lyrics]` placeholder, the model learns "absence → genre X" because missingness correlates with genre (instrumentals, undercovered artists). Best practice from the multimodal-missing survey literature (Wang et al. 2024 arXiv:2409.07825; Woo et al. MMP 2024):

1. **Learned mask token**, not zeros, not the mean of present lyrics — a trainable `mask_embedding` of the text dim, init `N(0, 0.02²)`.
2. **Binary availability indicator** concatenated to the fused representation so the model can condition on the signal explicitly.
3. **Modality dropout** during training (ModDrop, Neverova et al.): arXiv with p≈0.3, replace even present lyric embeddings with the mask token, forcing the audio branch to stay predictive on its own.
4. Report results with and without modality dropout, and on both full set and lyrics-present subset.

```python
import torch, torch.nn as nn
class LyricsAwareFusion(nn.Module):
    def __init__(self, d_audio=768, d_text=384, d_fuse=512):
        super().__init__()
        self.mask_embedding = nn.Parameter(torch.randn(d_text) * 0.02)  # NOT zeros,
        self.audio_proj, self.text_proj = nn.Linear(d_audio, d_fuse), nn.Linear(d_te
        self.has_lyrics_proj = nn.Linear(1, d_fuse)
        self.fuse = nn.Linear(3 * d_fuse, d_fuse)
    def forward(self, audio_emb, text_emb, has_lyrics, p_drop=0.3):
        if self.training:
            drop = (torch.rand(audio_emb.size(0), device=audio_emb.device) < p_drop)
            has_lyrics = has_lyrics & (~drop)
        text_in = torch.where(has_lyrics.bool().unsqueeze(-1), text_emb,
                              self.mask_embedding.expand_as(text_emb))
        return self.fuse(torch.cat([self.audio_proj(audio_emb),
                                    self.text_proj(text_in),
                                    self.has_lyrics_proj(has_lyrics.float().unsqueez
```

3. Task 2 — CLAP and MERT baselines on CPU (drop wav2vec2)

3.1 Model comparison

Model	Params	File size	Pooled dim	SR	Native ctx	Domain	Per-clip (batch 8 CPU)	To for 20
LAION-CLAP `clap-htsat-unfused`	~153 M	**~614 MB**	**512** (shared audio-text)	48 kHz	10 s	Music+env+captions	~200–400 ms	**~1. 2.2**
`larger_clap_music` (HTSAT-base)	~190 M	1.7 GB	512	48 kHz	10 s	Music-only	~500–900 ms	3–5 (sk
MERT-v1-95M	95 M	**~380 MB**	768	24 kHz	**5 s** (ideal for 3-s clips)	**Music (160k hrs)** huggingface	~80–200 ms	**~3 70**
MERT-v1-330M	330 M	1.3 GB	1024	24 kHz	5 s	Music	~300–600 ms	1.5
wav2vec2-base	95 M	378 MB	768	16 kHz	variable	**Speech only (LibriSpeech)** Hugging Face	~60–150 ms	~2 50
wav2vec2-large	317 M	1.26 GB	1024	16 kHz	variable	Speech	~300–600 ms	1.5

The recommended pair is LAION-CLAP `clap-htsat-unfused` (music-appropriate, joint audio-text, the de-facto music-captioning encoder in 2024–2025 ISMIR papers) plus MERT-v1-95M (music-specialist SSL, same 95M-param class as wav2vec2-base but trained on 160k hours of music at 24 kHz Hugging Face **with a 5-s context** huggingface huggingface **that perfectly fits your 3-s clips).** Wav2vec2 is rarely cited as a music baseline in ISMIR/EUSIPCO 2024–2025 — MERT has replaced it. If you include wav2vec2 at all, frame it explicitly as a *domain-mismatch control* showing speech-SSL doesn't transfer.

3.2 CLAP input-length handling

CLAP's `ClapFeatureExtractor` defaults to `max_length_s=10` with `padding='repeatpad'`. Hugging Face A 3-s clip is **repeat-padded up to 10 s internally by the processor** — no manual chunking needed. For the unfused checkpoint, the default `truncation='rand_trunc'` applies; for the fused checkpoint use `truncation='fusion'`. The output is always `(B, 1, T=1024, F=64)`. arXiv

3.3 Complete extraction code (three models)

```python
```

```python
# shared setup
import os, glob, numpy as np, torch, torchaudio
torch.set_num_threads(os.cpu_count()); torch.set_num_interop_threads(1)
device = "cpu"; BATCH = 8
paths = sorted(glob.glob("banglabeats/*.wav"))  # 20020 files, deterministic order

def load_mono(path, target_sr):
    wav, sr = torchaudio.load(path)
    if wav.shape[0] > 1: wav = wav.mean(0, keepdim=True)
    if sr != target_sr: wav = torchaudio.functional.resample(wav, sr, target_sr)
    return wav.squeeze(0).numpy()

# ── 1. LAION-CLAP (primary: joint audio-text, 512-d) ──────────────────
from transformers import ClapModel, ClapProcessor
clap = ClapModel.from_pretrained("laion/clap-htsat-unfused").to(device).eval()
clap_proc = ClapProcessor.from_pretrained("laion/clap-htsat-unfused")
out = np.zeros((len(paths), 512), dtype=np.float32)
with torch.inference_mode():
    for i in range(0, len(paths), BATCH):
        audios = [load_mono(p, 48000) for p in paths[i:i+BATCH]]
        inputs = clap_proc(audios=audios, sampling_rate=48000,
                           return_tensors="pt", padding=True)
        out[i:i+len(audios)] = clap.get_audio_features(**inputs).cpu().numpy()
np.save("clap_emb.npy", out)  # 41 MB

# ── 2. MERT-v1-95M (music-specialist, 768-d) ──────────────────────────
from transformers import AutoModel, Wav2Vec2FeatureExtractor
mert = AutoModel.from_pretrained("m-a-p/MERT-v1-95M", trust_remote_code=True).to(dev
mert_fe = Wav2Vec2FeatureExtractor.from_pretrained("m-a-p/MERT-v1-95M", trust_remote
out = np.zeros((len(paths), 768), dtype=np.float32)
LAYERS = slice(6, 10)   # middle layers are best for genre per MERT paper Table 2
with torch.inference_mode():
    for i in range(0, len(paths), BATCH):
        audios = [load_mono(p, mert_fe.sampling_rate) for p in paths[i:i+BATCH]]
        inputs = mert_fe(audios, sampling_rate=mert_fe.sampling_rate,
                         return_tensors="pt", padding=True)
        hs = torch.stack(mert(**inputs, output_hidden_states=True).hidden_states, di
        # (13, B, T', 768) → avg over layers 6-9 → avg over time → (B, 768)
        out[i:i+len(audios)] = hs[LAYERS].mean(0).mean(1).cpu().numpy()
np.save("mert_emb.npy", out)  # 61 MB

# ── 3. Wav2Vec2-base (OPTIONAL domain-mismatch control) ───────────────
from transformers import AutoFeatureExtractor
w2v = AutoModel.from_pretrained("facebook/wav2vec2-base").to(device).eval()
w2v_fe = AutoFeatureExtractor.from_pretrained("facebook/wav2vec2-base")
out = np.zeros((len(paths), 768), dtype=np.float32)
```

```
with torch.inference_mode():
    for i in range(0, len(paths), BATCH):
        audios = [load_mono(p, 16000) for p in paths[i:i+BATCH]]
        inputs = w2v_fe(audios, sampling_rate=16000, return_tensors="pt", padding=Tr
        out[i:i+len(audios)] = w2v(**inputs).last_hidden_state.mean(1).cpu().numpy()
np.save("w2v2_emb.npy", out)
```

All three cached `.npy` files together are under 250 MB. Once extracted, every subsequent clustering / VAE experiment is seconds of I/O.

3.4 CPU-speedup options (optional, only if you iterate a lot)

- **ONNX Runtime via Optimum** (`ORTModelForFeatureExtraction.from_pretrained(..., export=True, provider="CPUExecutionProvider")`) gives **2–5× on wav2vec2 and CLAP.** `Xenova/clap-htsat-unfused` already ships ONNX weights. MERT needs manual export because of `trust_remote_code`.
- **int8 dynamic quantization** (`torch.quantization.quantize_dynamic(model, {nn.Linear}, dtype=torch.qint8)`) gives another ~1.5–2× with <1 pt NMI/ARI drop on clustering.
- Combined: **~4–8× on 8-core x86**, pulling CLAP full-dataset extraction to ~15–25 min.

These are **nice-to-have, not required** — uncached fp32 extraction fits overnight.

3.5 Integrating into the paper

Drop-in K-Means baseline (the main Table 1 row):

```python
from sklearn.cluster import KMeans
from sklearn.preprocessing import normalize
from sklearn.metrics import normalized_mutual_info_score as NMI, \
                            adjusted_rand_score as ARI, silhouette_score

X = normalize(np.load("clap_emb.npy"))   # L2 for cosine-style clustering
km = KMeans(n_clusters=NUM_GENRES, n_init=20, random_state=0).fit(X)
print("NMI", NMI(y_true, km.labels_), "ARI", ARI(y_true, km.labels_),
      "Silh", silhouette_score(X, km.labels_, metric="cosine"))
```

Run with **5–10 random seeds** and report mean±std — ISMIR reviewers flag papers that report a single run. Add a **paired Wilcoxon signed-rank test** ($p<0.05$) between your VAE and each baseline (Vasquez et al. RecSys 2024 is the template).

VAE-on-CLAP/MERT (drop-in for MFCC): CLAP produces a single 512-d pooled vector per clip, so swap your spectrogram-conv encoder for a linear/MLP encoder. Before training, **PCA CLAP**

down to 64–128 dims (keeps ~95% variance, cuts VAE capacity needs substantially). MERT gives you either a pooled vector or a `(T'≈225, 768)` sequence you can feed to your existing 1-D conv VAE.

Expected numbers (indicative, from the literature — your exact values will vary):

Method	NMI	ARI	Purity
MFCC + K-Means (floor)	0.25–0.40	0.15–0.30	0.35–0.55
MFCC-VAE (your core contribution)	? — should beat MFCC+K	?	?
LAION-CLAP + K-Means	**0.50–0.70**	0.40–0.60	0.60–0.80
MERT-v1-95M + K-Means	**0.55–0.75**	0.45–0.65	0.65–0.82
Wav2Vec2-base + K-Means (control)	0.20–0.35	0.10–0.25	0.30–0.50

The narrative you want: your VAE is competitive with or complementary to foundation-model baselines, with orders of magnitude fewer pretraining data and params, and it works without a GPU. Even if foundation models beat the VAE outright, the paper remains publishable as *"here's how far a lightweight local approach gets on a new non-Western dataset, and here's the honest gap to SOTA foundation models."*

3.6 CLAP vs wav2vec2 for reviewer impact — final verdict

CLAP is unambiguously more compelling for a music-clustering paper, for three reasons:

1. **Domain match.** CLAP's training data (LAION-Audio-630K) contains music with captions; wav2vec2 saw only LibriSpeech. Recent ISMIR 2024 Audio Atlas demo (ETH Zurich) uses CLAP specifically for music clustering visualization.
2. **Joint audio-text space.** Because your paper is about *bilingual* music with lyrics, CLAP's shared embedding space with text is thematically aligned — you can do zero-shot genre probing as a sanity check.
3. **Community expectation.** The 2024–2025 music-representation baseline stack at ISMIR/EUSIPCO has converged on: MFCC → VGGish/MusiCNN → MERT → LAION-CLAP (+ MS-CLAP). Wav2vec2 has faded out of MIR papers except as an explicit speech-transfer control.

Concretely: add LAION-CLAP and MERT. Skip wav2vec2. If a reviewer asks for speech-SSL, add it to the camera-ready.

---

4. Related work you need to cite

The Bangla-specific MIR literature is small enough to enumerate exhaustively; the cross-lingual and VAE-clustering literatures are large, so focus on the strongest 2023–2025 anchors.

Cross-lingual / non-Western MIR (must-cite for framing):

- Li et al., *MERT*, ICLR 2024 (arXiv:2306.00107) — SOTA music SSL
- Plachouras et al., *Universal Music Representations? Evaluating Foundation Models on World Music Corpora*, 2025 (arXiv:2506.17055) — evaluates MERT/CLAP/Jukebox on non-Western corpora, directly motivates your setup
- Kalbag & Pandey, *CultureMERT*, 2025 (arXiv:2506.17818) — cross-cultural continual pretraining
- Kumar et al., *Music for All*, 2025 (arXiv:2502.07328) — documents that only ~5.7% of major music-dataset hours are non-Western; motivates Bangla work
- Frighetto et al., *Multi-label Cross-lingual Music Genre Classification from Lyrics with Sentence BERT*, 2025 (arXiv:2501.03769) — near-identical methodology on EN↔PT lyrics
- Dinkel et al., *GLAP*, 2025 (arXiv:2506.11350) — multilingual CLAP extension
- Lee et al., *Do Music Preferences Reflect Cultural Values?*, 2025 (arXiv:2506.13199) — CLAP + t-SNE + MANOVA template for cluster-statistical validation

Multimodal audio+lyrics (fusion design):

- Wu et al., *CLAP: Learning Audio Concepts from Natural Language Supervision*, ICASSP 2023 (arXiv:2206.04769); Wu et al., *LAION-CLAP*, arXiv:2211.06687
- Oramas et al., *Multimodal Deep Learning for Music Genre Classification*, TISMIR — canonical audio+text+image fusion
- Liu et al. (survey), *Multimodal Music Emotion Recognition*, 2025 (arXiv:2504.18799) — taxonomy of fusion
- Delbouys et al., ISMIR 2018 — audio+lyrics mood detection (partial-lyrics precedent)

Bangla-specific music work (disambiguate and extend):

- **Jibon, Alam, Rahman, *BanglaBeats: A Comprehensive Dataset of Bengali Songs for Music Genre Classification*, ICCIT 2023** — a published dataset with the exact same name (1,617 tracks, 8 genres, DistilHuBERT/Wav2Vec2 baselines). **Your 20,020-track version is ~12× larger; you must disambiguate the naming explicitly, either by renaming your dataset or by stating "extends ICCIT 2023 BanglaBeats" with provenance.**
- Rabbi, Hasan, Rahman, *Genre Classification from Bangla Music Lyrics: Dataset and Classifiers*, ICCIT 2024 — 10,500-lyric 11-genre corpus, BanglaBERT 92% — **direct lyric-side baseline**; also the dataset you should download

- Hossain et al., *BanglaMusicStylo*, 2018 — first Bangla-lyric stylometric dataset (2,824 lyrics)
- Sen et al., *Bangla Music Genre Classification Using Bidirectional LSTMs*, 2026 — 10-genre audio corpus (note preprint status)
- Mahajebin et al., *Mood Classification of Bangla Songs Based on Lyrics*, 2023 (arXiv:2307.10314) — 4k lyrics BERT-based baseline
- Ahmed et al., *BMGC*, NISS 2021; Mamun et al., 2019; Hasan et al., *BMNet-5*, 2022/2023 — earlier audio-only Bangla MIR works

VAE / deep clustering (core methodology):

- Kingma & Welling, *Auto-Encoding Variational Bayes*, 2014 — VAE
- Dilokthanakul et al., *GMVAE*, 2016; Xie et al., *DEC*, ICML 2016 — **must-cite** deep-clustering foundations
- Roberts et al., *MusicVAE*, ICML 2018 — seminal music VAE
- Caillon & Esling, *RAVE*, 2021 — raw-audio VAE
- Ronen et al., *DeepDPM*, CVPR 2022 — deep clustering with unknown K
- Vasquez et al., *Comparative Analysis of Pretrained Audio Representations in Music Recommender Systems*, RecSys 2024 (arXiv:2409.08987) — **the template for rigorous baseline comparison with statistical significance**
- Buisson et al., *Unsupervised Evaluation of Deep Audio Embeddings for Music Structure Analysis*, 2026 — nine-model unsupervised MIR benchmarking template

Novelty claim you can make honestly: no prior VAE or unsupervised deep clustering on Bangla music exists; no prior audio+lyrics multimodal Bangla music work exists; no 20k-scale Bangla music dataset exists. The prior landscape is supervised audio-only or lyrics-only classification on 1–3k-track datasets with 6–10 genres at regional IEEE venues.

---

5. What reviewers will expect in the extended paper

Based on the 2024–2025 ISMIR/EUSIPCO baseline-stack convergence (Vasquez et al. 2024, Buisson et al. 2026, Plachouras et al. 2025), a competitive submission needs:

Audio baselines (pick 3–5): MFCC+K-Means (floor) → VGGish or MusiCNN → MERT-v1-95M → LAION-CLAP → optionally wav2vec2 or MS-CLAP. Your MFCC-VAE sits between the floor and the foundation models.

Text baselines: paraphrase-multilingual-MiniLM-L12-v2 (current) → paraphrase-multilingual-mpnet-base-v2 or LaBSE (stronger) → BanglaBERT as a language-specific comparator. Reviewers increasingly expect multilingual sentence encoders with at least one language-specific counter.

Metrics: NMI, ARI, clustering accuracy (Hungarian-matched), purity — mandatory set. Silhouette for model selection. **5–10 random seeds, mean±std, paired Wilcoxon p<0.05** — papers reporting a single run are flagged.

Visualizations: t-SNE or UMAP of latent space (colored by genre *and* by language), confusion matrix, per-cluster purity table.

Cross-lingual-specific asks: per-language (EN vs BN) breakdown of every metric, and a **language-confound ablation** proving your clusters are not just separating English from Bangla. This is the single most likely reviewer critique for a bilingual clustering paper — address it preemptively.

---

6. One-week execution plan

Day	Task	Wallclock
1 AM	Download Kaggle dumps + BanglaMusicStylo + Rabbi 2024 corpus + Wikisource Bengali; dedupe to a single CSV. Pilot RapidFuzz match on 500 BanglaBeats tracks.	3–4 h
1 PM	Scale match to full dataset; spot-check borderline scores; export `lyrics_matched.csv` with `(track_id, title, artist, lyrics, source, bangla_ratio)`.	2–3 h
1 night	Targeted LyricsGenius sweep on top-500 unmatched popular Bangla artists (filter to `bangla_ratio>0.3`).	overnight
2 AM	Clean + encode lyrics with MiniLM (384-d); save `.npy` aligned to track IDs. Build mask-embedding fallback module.	2–3 h
2 PM–night	Extract **LAION-CLAP** (~1.5 h) and **MERT-v1-95M** (~45 min) embeddings for all 20,020 clips in parallel if cores allow; save `.npy`.	~3 h
3	K-Means baselines (10 seeds each), metrics, t-SNE/UMAP figures, per-language breakdown. Compile Table 1.	full day
4	Retrain VAE on CLAP/MERT features as additional experiment; lyrics-aware fusion ablation with modality dropout.	full day
5	Write related-work section; integrate Plachouras, CultureMERT, Rabbi 2024, Jibon 2023, MERT, LAION-CLAP citations. Draft Table 2 (lyrics-present subset ablation).	full day
6	Paper extension, figures, language-confound ablation, statistical significance tests.	full day

Day	Task	Wallclock
7	Polish, proofread, double-blind anonymization check, submit to ISMIR 2026.	

Conclusion — the shape of the extended paper

The workshop paper's one-sentence contribution ("VAE-based unsupervised clustering of a large hybrid EN-BN music dataset") becomes a three-sentence conference contribution: **(1) a 20,020-track hybrid EN-BN music corpus that is ~12× larger than the existing BanglaBeats and the first at ISMIR-scale; (2) a VAE-on-MFCC clustering baseline benchmarked honestly against LAION-CLAP and MERT-v1-95M foundation models, including a wav2vec2 domain-mismatch control and statistical-significance testing; (3) a lyrics-aware fusion ablation on a ~25–40% lyrics-matched subset, using real Bangla lyrics from Kaggle/BanglaMusicStylo/Wikisource, with a learned-mask-token fallback and modality dropout to prevent label leakage.** That contribution set — dataset + foundation-model benchmarking + careful multimodal ablation — is exactly what ISMIR 2026 reviewers will accept as a workshop-to-conference extension.

The core judgment calls the analysis changed: **(a) Kaggle bulk dumps dominate Genius as a primary Bangla-lyrics source;** Genius coverage is far weaker than its English reputation suggests. **(b) MERT replaces wav2vec2 as the music-SSL baseline of choice**, and LAION-CLAP is the music-text encoder of choice — the 2024–2025 literature has converged here. **(c) The "is my coverage enough?" question has no fixed answer; frame the lyrics work as a subset ablation with transparent per-set metrics and the mask-token fallback, and you can publish with as little as 10–25% coverage. (d) ISMIR 2026 is the only viable venue given April 2026 timing**; EUSIPCO and SMC are closed.